

How Big Tech does Quality Assurance (QA)

Most Big Tech companies have no dedicated SDET, QA, or tester roles. How do they produce quality software? A look into how Microsoft, Google, Meta, Apple, Amazon, Uber and Netflix do it.



GERGELY OROSZ

SEP 19, 2023 • PAID



81



6



4

Share



👋 Hi, this is Gergely with a 🔒 subscriber-only issue 🔒 of the Pragmatic Engineer Newsletter. In every issue, I cover challenges at Big Tech and startups through the lens of engineering managers and senior engineers. To get articles like this in your inbox, weekly, subscribe:

Programming note: *I'll be traveling for 3 weeks in November, heading over to the US – SF and NYC –, with less than usual time to write. I'm looking for guest writers to help out during this time. If you are an engineer or engineering manager with hands-on expertise to share, please [indicate your interest here](#).*

In this issue, we take a look at how major companies go about producing quality software, covering:

1. Microsoft
2. Google
3. Meta
4. Apple
5. Amazon
6. Uber
7. Netflix

I gathered details on the approach to QA from current or former engineers and engineering managers at each of these companies to confirm findings – *thank you to everyone who helped*. We have a lot to go through, so let's get to it:

1. Microsoft

The Redmond, Washington-based tech giant has played an outsized role in the development and importance of quality assurance across the industry. Microsoft was the first major company to come up with a specialized testing role which went well beyond manual testing.

The SDET role

The SDET (Software Development Engineer in Test) role was one that Microsoft pioneered across the tech industry. They are *software engineers* who focused on writing automated tests; building and maintaining testing systems. The only difference between an SDET and a software development engineer (SDE) is that SDETs didn't generally write production code: they wrote test code, working in the same team as SDEs.

I could not trace the *exact* introduction of the role, but it was most likely in the 1990s. For example, here's [a post from 2004](#) from a member of the Microsoft Exchange team explaining what it means to be an SDET in their organization:

"An SDET is a developer who works in a test team and not a development team. The SDET has a keen sense of a tester yet loves writing code, and lots of it.

The SDET enables the test team with the tools and processes that need to be in place for the testers to do what they do best... test the living daylights out of the product and find as many bugs as possible before it goes to the market.

An SDET has the ability to analyze the functionality and architecture of a Product and thus design and implement tools that help testers test it.

The SDET enjoys short project life cycles, designs and implements many tools and test frameworks in a single year, uses the latest technology and has plenty of room for innovation.

Though product quality is a prime concern, the SDET doesn't have the stressful days that developers have during the end of a product life cycle. In layman's terms... an SDET backside is rarely on the line :)"

Microsoft had a formal career path for the SDET role. From the same post:

"There's plenty of room for growth in [the SDET position.] If you love doing what you are doing as an SDET then you can grow to become a Test Architect. If you want to get involved in management, then you can progress towards becoming an SDET Lead and then Test Manager.

If you want to just code and not be involved with testing then you can take the path of becoming a developer. Many people take this path. If you realize that your heart belongs to testing, then you can become a tester."

A 2:1 SDE:SDET ratio was common across Microsoft until around 2014. It was the case on my team at Skype for Xbox One, in 2012, when the team was formed. Here's how our team was composed, based on headcount:

- 12x SDEs (software development engineers)
- 6x SDETs (software development engineer in test)
- 2x PMs (product managers)
- 1x EM (engineering manager)
- 1x SDET lead

On our project, the SDET team owned all parts of testing:

- Manually verifying that features devs built, worked as expected, including edge cases we might have not considered
- Building integration and end-to-end tests to automate checks
- Creating manual tests plans, and executing before major milestones
- Being involved during the planning phase of features, bringing ideas on edge cases and how the work would be validated
- Building solutions for tricky problems, such as how to do reliable performance benchmarking for our Skype product on the Xbox hardware

Unit tests were a source of tension, early on. Who should write them? Several experienced developers came from gaming, where developers typically don't write automated tests, and their view was that any automation – including unit tests – should be done by the SDET teams. *We cover more on how games are built in [Game development basics](#), and go very hands-on in the issue, [Building a simple game](#).*

Those of us who'd previously built applications, or did test driven development (TDD,) felt that this approach was wrong, and that developers should write their own unit tests because unit tests and the code are tightly coupled. I was in this camp.

Having dedicated SDETs made it a tempting option for developers to "outsource" the writing of unit tests. I'm just going to say it now: without an SDET team, the question of who writes unit tests would have not been up for debate: we developers would have *had* to write them. This is a recurring debate I've seen in every team with assigned SDETs. Surprisingly, even this year I heard of a Silicon Valley-based company where a developer team has the test team write unit tests!

In our case, we settled on developers writing unit tests, with the SDET team doing everything else. This approach worked well enough, but there were a few memorable features of this setup:

- **An "us and them" dynamic that created division.** When us developers finished a feature, we handed it over to an SDET, who usually found issues, so the feature came back to developers to fix. This felt annoying to devs, as it created work we might have not accounted for. Over time, it started to feel like there were two teams, with different goals, which didn't always row in the same direction.
- **Ticket ping-pong between dev and test.** I finish work on my feature and send it over to test (ping). The tester finds a bug and sends it back to me the next day (pong). I fix the bug, and send it over again in a few days (ping). The tester finds another bug and sends it back to me... and this back-and-forth happened more times than I'd be willing to admit!
- **Devs were good at building complex systems, and could help SDET a lot.** Our SDET team had been building an integration testing system for months, and progress was slow. Our team really needed this system, as manual tests were taking too long. Finally, one of the senior engineers proposed that devs join in to help build this system, *as one team*. Two weeks later, with the lead of experienced devs, a system was up and running. This got me thinking; would the team not work better without the dev/test division? We had just proved it did.
- **The elephant in the room: some devs looked down on the SDET role.** Although not everybody did, it was clear that many devs regarded SDET work as less challenging than their own. SDETs also knew they could have better career options by switching to a dev role.

Well, it turns out they didn't have to wait that long for advancement.

Retiring the SDET role

Early 2014, I joined a new team called Skype for Web. This team was different from most teams at Skype, as they shipped a new version of the software every day, not every month.

The team consisted of 6 SDEs and 3 SDETs, on paper. In reality, we were 9 SDEs, thanks to the engineering manager and the test lead who quietly decided it made zero sense to have a dedicated test role, when we shipped new features every day. I write about this change that the team's leadership didn't broadcast, in the issue [How Big Tech runs tech projects and the curious absence of Scrum](#):

"When I joined the Skype for Web team, we initially did two-week sprints, and followed the usual Scrum processes. We also had a split of software engineers and QA engineers. However, our shipping pace was every two weeks, but we wanted to ship more frequently.

The first thing we did was make QA a part of engineering. In the "old world", an engineer would finish their work, check into their branch, update a ticket, and let the QA know it's ready for review. The QA would take this ticket a day or two later, review it, and reopen the ticket if they found issues. This was a long delay.

We made a quiet, unofficial, change where all SDETs built production software as well, and all software engineers became responsible for testing their own code. Now, we no longer had to wait days for feedback before shipping code to production. However, the bi-weekly sprints and the numerous Scrum rituals became the next problem."

We became a lot more productive by removing the SDET role from our team! SDETs still focused mainly on testing-related work, but also picked up development tasks. Just as importantly, we paired a lot! I remember pairing with SDETs to build a feature. I was good at thinking about how to make something work, and the SDET was really good at pointing out edge cases I hadn't considered. When it came to debugging, the resourcefulness of SDETs surprised me.

On most teams across Microsoft, SDETs spent a lot of time manually testing things, and writing integration tests. But on our team there was very little manual testing, and we all built integration testing *infrastructure*, and monitoring *infrastructure*. When a developer or an

SDET picked up a piece of work, they wrote all tests – unit and integration – which made sense.

The best part of this change was that there was no more “us vs them.” Arguments ceased about whether to fix a bug which an SDET had discovered, as now we did our own testing and fixed bugs which we discovered, before shipping to production.

Web teams across Microsoft started to quietly remove the SDET role. Back in 2014, our web team in the London Skype office felt ‘special,’ as the only other teams to merge the SDET function were web-based teams, of which there were not many. On every other team, SDETs kept working the way they always had.

However, it wasn’t only web teams within the Skype division which merged these roles for better efficiency. Web teams independently came to the realization that merging SDET and dev roles made them move faster, and so this happened across all of Microsoft!

In the middle of 2014, Microsoft formally retired the SDET role and introduced the SE role. The inspiration was apparently a larger web team at Microsoft, Bing. [From Ars Technica](#), in 2014:

“At Bing, the task of creating programmatic tests was moved onto developers, instead of dedicated testers. QA still exists and is still important, but it performs end-user style “real world” testing, not programmatic automated testing. This testing has been successful for Bing, improving the team's ability to ship changes without harming overall software quality.”

In July 2014, Microsoft announced that they will execute their largest layoffs to that date, letting go 18,000 staff of the 127,000 employees at the company. 12,500 of the cuts were for the Nokia division. As part of this layoff, a large number of SDET roles were also eliminated. This happened around the same time as the SDET role was announced to be retired, and existing SDETs needed to move over to the software development engineer (SDE) track over the next several months. The SDE role was also renamed to SE – Software Engineer.

How did this transition work out? From what I gather, it went fine. The change made a lot of sense for teams that ship on a daily basis. And teams within Microsoft that ship weekly or monthly are increasingly rare, as Microsoft also leans into the software-as-a-service (SaaS) model. Of course, Microsoft continues to be a vendor for the Windows operating system

family, and the Surface tablet. These are both areas where the approach to quality needs to be different to that of SaaS products.

A good account of the change came from the Visual Studio Team Services team in 2017, three years after this change. Reflecting on it, [Brian Harry](#) — currently Technical Fellow at Microsoft — [wrote](#):

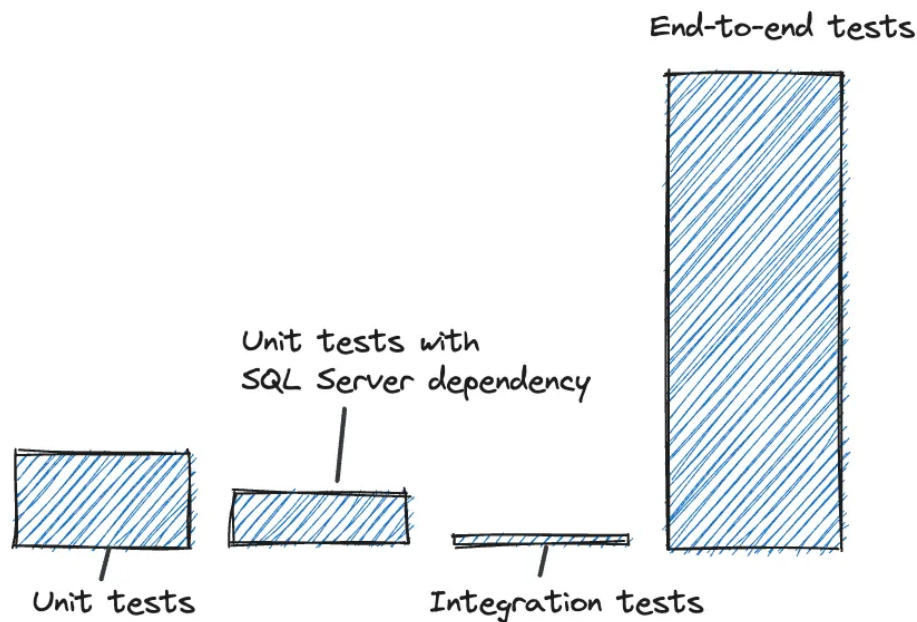
"Two years ago [in 2015], we had 10's of thousands of tests. They were written by 'testers' to test code written by 'developers'. While there were some advantages of this model – like clearly measurable and controllable investment in test, expertise and career growth in the testing discipline, etc., there were also many disadvantages – lack of accountability on the developers, slow feedback cycles (introduce bug, find bug, fix bug), developers had little motivation to make their code "testable", divergence between code architecture and test architecture made refactoring and pivoting very hard/expensive, and more. (...)

Full testing would take the better part of a day to run, many more hours to "analyze the results" to identify false failures, and days or weeks to repair all the tests that were broken due to some legitimate change in the product. (...) Two years ago, we started on a path to completely redo testing.

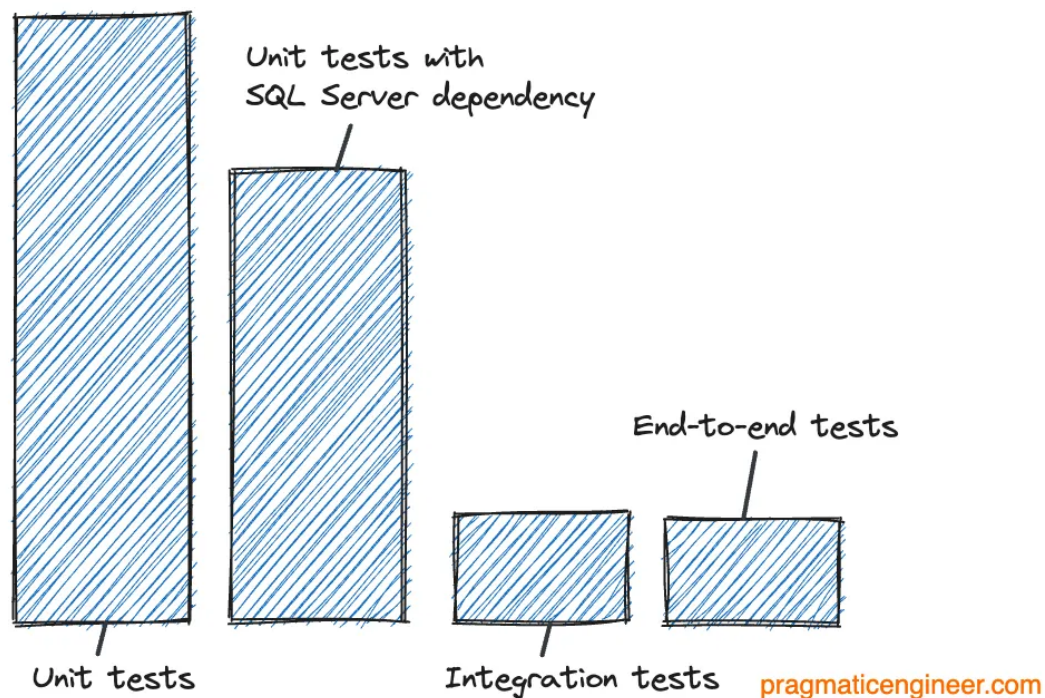
We combined the dev and test orgs into a consolidated 'engineering' org. For the most part, we eliminated the distinction between people who code and people who test. That's not to say every person does an identical amount of each, but every person does some of everything and is accountable for the quality of what they produce. We also set out to completely throw away our 10's of thousands of tests that took 8 years to create, and replace them with new tests that were done completely differently."

This team took stock of the type of tests they had in place and decided they didn't like that there were few small unit tests, but lots of complex and hard to maintain end-to-end tests. So they changed this:

Before merging dev & test (2015):

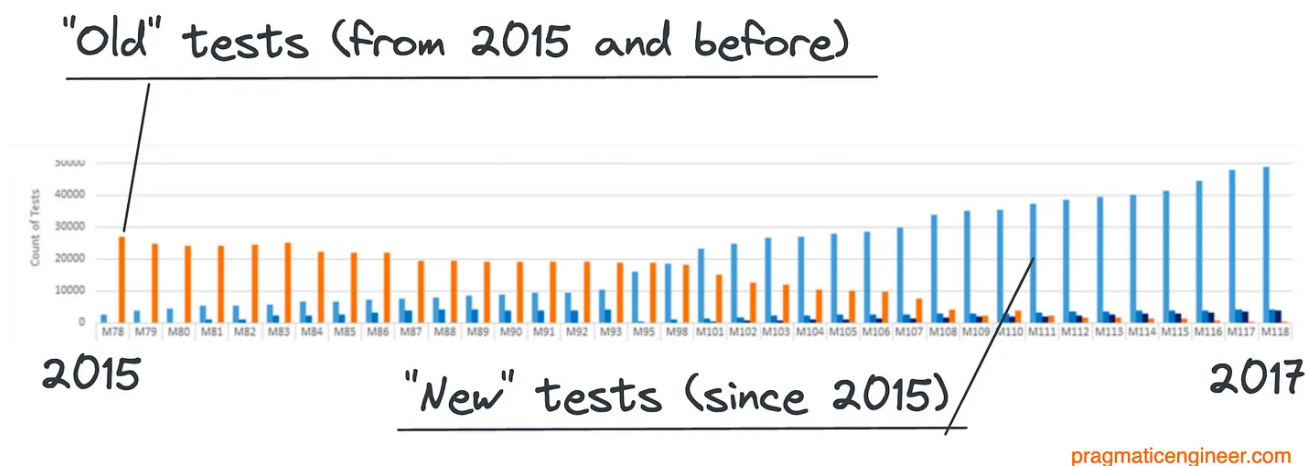


After merging dev & test (2017):



The change in the number and type of tests the Visual Studio Team Services experienced after merging the dev and the test teams. Before the merge: end-to-end tests dominated, but unit and integration tests were rare. This flipped after the merge. Data source: [Microsoft Dev Blogs](#)

Here's another visualization on how the team's tests changed over a two-year timeframe:



In 2 years, almost all "old" tests from when test was separate from dev, were gone. The new tests became more granular as well. Data source: [Microsoft Dev Blogs](#)

So, was all this work worth it in the end? According to Brian, yes it was. Writing at the time, he said:

"We are starting to reap the benefits in improved quality, agility and engineer satisfaction."

2. Google

Google is a big enough company that it's not possible to generalize across all divisions, like Search and Docs, and hardware divisions like Nest, and so on.

However, across most software teams, engineers are responsible for quality, for which they use unit tests. Teams can build other, automated tests as necessary, like integration, end-to-end, load tests, and so on.

EngProd is a specialized division within Google responsible for engineering productivity infrastructure. Here's how the company [describes](#) it:

"[EngProd is] a data-driven engineering discipline focused on optimizing the engineering process so Google can deliver amazing experiences to our users, faster."

The focus of this organization is:

- **System analysis:** identifying gaps in the engineering process, where new tooling could improve engineering excellence and speed
- **Instrumentation:** it's hard to improve something that's not measured. Google obsesses over metrics, and starts with instrumenting tools and processes so any change in efficiency can be measured
- **Tools and infra:** building infrastructure such as testing, CI/CD, and everything else that can help engineers move faster, with higher quality
- **Working within product teams:** EngProd engineers can embed themselves in product teams, and help them work faster and more efficiently

On anonymous network Blind, a Googler working in EngProd [described](#) the division's focus on testing like this:

"One common topic where EngProd intervenes is testing, testing frameworks and testing infra. Engineering teams don't really want to write new testing frameworks, they don't have the bandwidth, or expertise. But as their products evolve, making incremental changes to the way testing works can actually do more harm than good.

EngProd teams can help by owning the testing strategy, advising on architecture to optimize testability and building new tools as needed. And as product teams don't need to spend too much time thinking of these problems, they can move faster.

There are lots of other problems besides testing that kind of work the same, where a few engineers can address a problem that impacts hundreds or thousands of others, and make significant changes."

Test Engineering is a role at Google, which is typically for hardware teams. The company describes this role like this:

"Test Engineers at Google aren't manual testers -- you write scripts to automate testing and create tools so developers can test their own code. As a Test Engineer, you navigate Google's massive codebase, identify weak spots and constantly design better and creative ways to break software and identify potential problems. You'll have a huge impact on the quality of Google's growing suite of products and services."

Google hires Test Engineers for teams like:

- Augmented reality products
- Fitbit products
- Pixel smartphone
- Wireless and networking products

Google Cloud hires for Hardware Performance Test Engineers, and System Test Engineers: both of these with a hardware focus.

Employing testers on a contract is a thing at Google, case-by-case. This is common for complex projects, or organizations. For example, I talked with a senior engineer whose organization employed a few contract testers in an organization where 30+ teams were rolling out 50+ features to production, per month.

Contract/freelance testers are usually hired to take on:

- Smoke testing before a release. This is also called verification testing, or confidence testing. It's often manual, although manual steps can be automated
- Write end-to-end tests before a first or a major release

What contract testers do not do is write unit or integration tests, as engineering teams own these.

A book that is a helpful overview on how Google *used* to test, a decade ago is the [How Google Tests Software](#) book by James Whittaker, Jason Arbon and Jeff Carollo, published in 2012. This book introduces the Software Engineer in Test and the Test Engineer roles:

"Google Software Engineer in Tests (SETs) are test developers, responsible for assisting SWEs with the unit test portion of their work and also in writing larger test frameworks to assist SWEs in writing small and medium tests to assess broader quality concerns.

Google Test Engineers (TEs) are user developers, responsible for taking the users' perspectives in all things that have to do with quality. From a development perspective, they create automation for user scenarios and from a product perspective, they assess the overall coverage and effectiveness of the ensemble of testing activity performed by

the other engineering roles. It is not utopia, but it is our best attempt at achieving it in a practical way where real-world concerns have a way of disrupting best intentions in the most unforeseen and unforgiving way."

3. Meta

Meta has not had a formal QA or SDET role, to the best of my knowledge. Software engineers (SWEs,) Production Engineers (PEs,) Solution Engineers (SEs,) and Enterprise Engineers (EEs) handle all testing for the software their teams own. *We cover these roles in more detail in the issue [Inside Facebook's engineering culture](#).*

While Meta [used to](#) have the reputation of not being big on unit testing – and relying on a world-class automated rollout/rollback system, instead – this seems to be changing in organizations like infra. Within infra, several teams use a test-driven-development (TDD) approach to software development, writing tests first, and then production code. For more mature products, it's expected that test cases cover all the changes.

Each team decides its testing approach. This is important to emphasize. For example, when Meta built the Threads app in five months with a small team, they [did not worry about automated testing for the initial release](#). Finding product-market-fit was much more important.

DevInfra is an organization within Meta that's very similar to EngInfra at Google. Here's how the company describes the focus of this organization:

"DevInfra owns most of the coding life cycle at Facebook, from the time code leaves an engineer's mind, until it reaches the people who use our apps. Our mission is to increase developer efficiency so we can continue to ship awesome products quickly.

We lead the industry by building innovative developer tools and automation infrastructure that are reliable and fast, ensuring that every second of engineering time is spent on the things that matter."

The DevInfra team does the heavy lifting, both for building developer tooling, as well as for test infrastructure. Here are a couple of neat solutions this team built:

- [Autonomous testing of services](#), done at scale. This is an extension on top of Meta's integration testing infrastructure

- [A vacuum testing approach for resilience](#): verifying disaster recovery in complex environments
- [Automatically detecting deteriorating tests](#). Aka: "how do you test your tests?"

Dedicated QA does exist within Meta. However, I have only heard of this role in relation with hardware products, within Reality Labs.

Meta's approach in shipping is pretty unique, across the industry. This is especially true for the core Facebook product, which is a sophisticated approach few other companies follow. Code rolls out through 4(!) different environments in an automated way, with automated rollbacks when things look off. *We touch on more details in the issue [Shipping to production](#).*

4. Apple

Among all of Big Tech, Apple is the one that's maintained the most focus on dedicated QA. To the best of my knowledge, it is also the last remaining Big Tech company with a Director-level QA role.

Within the company, many product teams have dedicated QAs who do both manual and automated testing. Apple also has an organization focused on delivering the operating systems (MacOS, iOS.) This team operates on a 365/24 scale across the globe; meaning the organization is available all day, every day of the year. The focus of this organization includes triaging reported bugs by verifying that they occur, and routing them to the right teams to resolve.

To get a sense of expectations in this role, I looked at some recently listed QA engineer positions; one for the [Software & services organization](#), one [Safari & Webkit team](#), one [for Apple Pay](#); and one [for Generative AI](#) (yes, Apple is also active in this field!) Common expectations of these roles include:

- Write code using the language the team uses. This can vary by team; those mentioned across these positions were Python, Swift, Objective C, JavaScript, and scripting languages
- Be familiar with test automation, with tools like XCTest and CI/CD
- Familiarity with testing approaches like unit, integration, performance and functional testing

- Possess qualities helpful for QA engineers: good debugging skills; creativity; organized; motivated; and a good communicator
- Enthusiasm for user-focused, high-quality products

Most teams expect QA engineers to develop test plans, for writing test cases, defining and tracking test coverage, evaluating readiness to ship, and assessing the risk of bugs discovered.

The [generative AI quality engineer](#) merits a special mention. This field is different to traditional software engineering, and so are the ways of testing GenAI models. Quality engineers are expected to have strong programming skills using standard ML tools – like PyTorch, Tensorflow – and languages like Python, Swift, C++. Hands-on experience using large language models (LLMs) is a must. This position feels like it's looking for an ML engineer with a passion for testing. *A lot of this overlaps with the toolset we cover in the issue [The machine learning toolset](#).*

Are the focuses on QA and the quality of product connected? If you had me answer which company releases the most polished products from among Microsoft, Meta, Google, Amazon, Netflix, and Apple; my answer is Apple, without hesitation. Could it be coincidence that Apple is the one that seems to keep investing most in a standalone, dedicated QA role?

It would be easy to assume causation – that Apple has high-quality products thanks to its dedicated QA role – but I'm a bit less certain this is the case. My sense is that Apple has held on to the QA role due to having a very heavy hardware focus, unlike the other four companies. However, for digital-only product quality, I am not so sure this holds. Both Apple TV and Netflix are polished products that are very pleasant to use, and Netflix built its offering without a dedicated QA function, while Apple does have one.

5. Amazon

Amazon has an SDET role, which not only includes a definition, but also a promotion framework that goes up to the principal level (L7.)

SDETs tend to operate in more specialized contexts, such as:

- Building and maintaining testing frameworks or core systems
- Working within specific, complex front-end systems

- Working with physical devices

The majority of teams are composed of a PM, an SDM, and SDEs. On these teams, SDEs are expected to perform testing of their own products. There are central teams that use the testing frameworks used by other teams.

It's down to local teams to decide if they need a dedicated SDET. This is similar to how teams decide if they need data science roles, or a TPM.

There isn't a central pool that I know of, but there are central teams which build the tools everyone else uses, such as test frameworks, etc.

As I browsed job openings [for SDET roles](#) at Amazon, the majority of these positions were for hardware-related products, and a minority for products with iOS, Android and Fire TV/tablet apps:

- Amazon Appstore developer experience
- Amazon's Automotive division (they have one, and are developing innovative automotive-shopping experiences)
- Amazon Kuiper: the company's satellite broadband network
- Amazon Photos (with apps on iOS, Android, Fire Tablet, Fire TV and the web)
- Amazon Transportation (a fulfillment network)
- Blink outdoor camera
- Fire TV a Tablet
- Ring video doorbell
- within the Amazon Consumer Robotics group; the Blink outdoor camera

Amazon's Lab 126 is a division with a strong hardware focus. As such, the company employs dedicated SDETs, as I confirmed with an engineer in this division.

My sense is that while any team can request a dedicated SDET, it's mostly hardware-related teams, or those on a slower release cadence – like releasing apps – that do so. At the same time, it's important to note Amazon is the only other major company other than Apple with a dedicated and well-defined QA/SDET career path.

6. Uber

Uber has no equivalent of SDET, QA, or similar roles. All product and platform teams tend to consist of a product manager, engineering manager, and software engineers. The team is responsible for the quality of the products they build. How this is done is down to the team deciding. On most teams I worked on – and ones I knew of – product managers were advocates for customers – and quality – and many of the engineers [were product-minded](#), putting product quality and user satisfaction high on their priority list.

Platform teams build test infrastructure for various verticals that include testing frameworks, as well as CI/CD setups which run these tests. As an example of what these teams build as testing infrastructure, here's a glimpse of Uber's Pixel device farm, built and maintained by Uber's mobile platform team:



Ty Smith
@tsmith



Our shiny new Pixel device farm at [@UberEng](#). One of the ways we help scale development and automated testing for hundreds of Android engineers and as many commits per day.



11:30 PM · Jun 3, 2022



101



507



3,643



459



A subset of Uber's mobile device testing lab. Source: [Ty Smith / X](#)

Given Uber is a mobile-first company, and that mobile releases are done on a weekly basis, there's additional, manual testing done here. This is a mix of manual tests the teams take on – called sanity tests – as well as several other forms of testing:

- 'Sanity testing:' developer teams executing manual tests that they have defined. These are tests that the developer team either cannot automate, or decides it's not worth automating
- Employee dogfooding of the latest, unreleased app version

- Beta programs
- External testing, often done with external testing vendors

I joined Uber a couple of years after having worked at Skype/Microsoft, directly with SDETs. To me, not having dedicated QA only felt like an issue during the weekly 'sanity testing' chores our team had. Eventually, we got frustrated with manually running these tests, and so built end-to-end tests for some of them. For those we could not automate – due to needing end-to-end test payment paths – we made a case for hiring an external company that was able to crowdsource specialized testing, where testers needed to be in very specific countries, using regional payment methods.

The platform team has built other neat tools which help engineering teams validate the quality of their work, while moving fast:

- [Rolling out multi-tenancy across microservices](#), to allow for safe testing in production
- [Short-lived application testing environments](#) to create and use on demand, ephemeral testing environments

Having no dedicated testers means everyone else has to care more about product quality. Back in 2016, telling people that Uber had no dedicated testers would draw strange looks from developers used to working with QA folks. Many of these people assumed that without QA, the focus on quality would fall between the cracks.

But here's what happened, instead:

- **Designers** were very much invested in how well the experience worked. They stress tested early builds, and gave feedback
- **Product managers, engineering managers, and engineers** would have a sense of pride in building and releasing their work
- **Customer support** surfaced customer complaints and channeled them to engineering teams via the product operations function, or product managers
- **Dogfooding the product** and making it very easy to flag issues worked pretty well, given Uber had more than 10,000 people dogfooding the app! This also has the opportunity for internal escalation. I still remember that bug submitted by Uber CEO Travis Kalanick, that my team had to handle – which we did as a priority!

- **Business metrics** were things teams kept a real time pulse on. When a business metric started to go down – on my team, this was the percentage of successful payments – we would dig into the cause and whether it was a functional thing, or a user experience one.

For more details on why and how Uber created platform teams, see the article [The platform and program split at Uber](#). I cover more details on mobile automated and manual testing testing in my book [Building Mobile Apps at Scale](#).

7. Netflix

I asked around within Netflix whether there is a concept of QA or SDET roles, and there is not. The company, like Uber, leaves engineering teams to ensure the quality of their work is up to customer expectations. This responsibility will be split between the product manager, engineering manager, and the developers.

The only QA roles I've observed are in the relatively new Netflix Games Studios group, where the company is hiring a [QA Manager](#), a [senior test engineer](#) and a [senior QA analyst](#)

While there is no dedicated role for testing, Netflix has had a major impact on the industry by how it approaches quality.

Chaos monkey testing originates with Netflix. In 2010, the company published a blog post which [first explained](#) the concept:

"One of the first systems our engineers built in AWS is called the Chaos Monkey. The Chaos Monkey's job is to randomly kill instances and services within our architecture. If we aren't constantly testing our ability to succeed despite failure, then it isn't likely to work when it matters most – in the event of an unexpected outage."

Netflix [open sourced](#) its Chaos Monkey tool in 2016, and the concept of chaos monkey testing has spread to other companies since. I would argue that Chaos Monkey was one of the approaches that kickstarted the [Chaos engineering principles](#), first published in 2015. There's now a [Chaos community](#) dedicated to this topic. Perhaps it wouldn't exist without Netflix!

A/B testing and experimentation is at the heart of the company, as discussed in the 2016 video about [Design testing at Netflix](#). In it, Anna Blaylock from the Netflix design team,

shares how Netflix learned that what users ask for, is often not what they really want. The company's metrics-based approach – of paying attention to what users do when a change is rolled out – helped Netflix build features that resulted in more product usage, and hopefully, more satisfied users.

Netflix has built lots of tools to measure the quality of its product, including a tool called Video multimethod assessment fusion ([VMAF](#)), which is a way to measure the *perception* of streaming quality. For example, in the images below, the top and bottom images have the same objective quality, but to the human eye, the top fox image feels significantly worse quality:

These images are the same "absolute" quality



These images are also the same "absolute" quality
So why does the fox feel like it's "lower" quality vs the crowd one?

pragmaticengineer.com

The Netflix engineering team built a tool to measure the perception of video quality. In the above image, the fox feels like it's a lot worse quality for the top image compared to the crowd: but they are the same, in an absolute sense!

Image source: [Netflix tech blog](#)

Like Uber, Netflix also does [testing in production](#) and its engineering team built a neat way to test on mobile devices, [in an automated way](#). These are just some of the examples I came across, there are other testing tools and improvements built by their team.

Takeaways

It was Microsoft that first established the SDET role in the 1990s, and yet this giant of tech discontinued the position in 2014, as more and more of its teams moved to a daily release cadence, instead of monthly or quarterly ones. Now, it is teams themselves which own quality, and software engineers build plenty of automation to ensure what they release works as it should.

Apple and Amazon are the only two Big Tech companies that still have a dedicated Quality Engineer (Apple,) or SDET role (Amazon,) in place. While the benefits of QA are obvious for Apple's hardware products, I have to wonder how much of a visible difference Apple's QA approach plays with software. At Amazon, software-only teams rarely have SDETs, though they can request them for dedicated help.

Google and Meta have an organization focused on developer productivity – EngProd (Google,) and DevInfra (Meta) – and these organizations also build testing tools.

Uber and Netflix have never had dedicated QA or SDET folks, but each has invested massively in building testing infrastructure. The Chaos Monkey tool by Netflix has spread across the industry, while at Uber, platform teams keep coming up with better ways for engineers to validate their work quickly in a production-like environment; sometimes right in production!

All of Big Tech invests heavily in ensuring the quality of their products: but this is not usually in the form of manual testing, or manual testers. This group pushes heavily to measure the quality of their products. Once they can measure, they automate the measurement, and build automated feedback loops and tests which keep track of where these quality indicators are.

Within Big Tech, it is increasingly only hardware products and software products on a slow release cycle, that have dedicated QA staff assigned to them. At the same time, software engineers bear a lot more responsibility for ensuring the quality of the products they build, than at more 'traditional' companies with dedicated testers.

I hope this overview illustrates why these choices make sense for each firm, and the additional approaches these organizations take to ensure high product quality. As always, resist the urge to copy what another company is doing just because it is 'famous' like Google or Apple.

In a follow-up issue, we'll cover testing approaches across a wider range of companies, and observe that the QA role is present at many companies, and growing in importance in some places. We'll also analyze how the QA role is changing across the tech industry, and why.

Related issues, which overlap with testing:

- [How Big Tech runs tech projects and the curious absence of Scrum](#). How projects are organized does have an impact on how straightforward – or not – it is to do manual testing on a regular cadence
- [Healthy oncall practices](#) – while not all Big Tech companies have a dedicated QA function, engineers are almost always oncall, and this forces some more focus on quality
- [Shipping to production](#)
- [Dealing with a low-quality engineering culture at Big Tech](#)
- [What is a senior software engineer at Big Tech?](#)



81 Likes · 4 Restacks

6 Comments



Write a comment...



Adam Sep 20 · edited Sep 20 ❤️ Liked by Gergely Orosz

> The elephant in the room: some devs looked down on the SDET role. Although not everybody did, it was clear that many devs regarded SDET work as less challenging than their own. SDETs also knew they could have better career options by switching to a dev role.

This frustrates me and causes me to second-guess my path as an SDET. I've often found that doing good SDET work runs parallel to doing good dev work; testing complex systems usually involves building and deeply understanding complex systems. The work is challenging and rewarding, but frequently feels disrespected, and my possible career paths possibly stunted due to this perception of SDET work being lesser. The reward loop also gets elongated. The amount of bugs you catch is an imperfect measure of impact, but it's hard to feel useful if your work doesn't immediately rack up a big bug number, which can easily happen depending on what you test.

♡ LIKE (3) 💬 REPLY ↗ SHARE

...



Aadil Maan Writes BR: Technical Program Manager's... Sep 20

What made Apple even more unique is that the Central QA team reported to the PMO leader instead of any engineering team. There was great emphasis on manual testing which does a better job of catching edge cases and thus the more polished feel and look of Apple software. QA leads were embedded and engaged from the very start of feature development which made them first class citizen rather than a checklist item which is what felt like at Google for projects I worked on.

♡ LIKE (1) 💬 REPLY ↗ SHARE

...

4 more comments...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing